

Testes de Software

Implementando Padrões de Teste (Test Patterns) Laboratório CheckoutService — test-pattern

Aluno: Sthel Felipe Torres

Matrícula: 1464964

Fork: <https://github.com/SthelTorres/test-pattern>

1. Padrões de Criação de Dados (Builders)

1.1 Por que o CarrinhoBuilder em vez de um CarrinhoMother?

O padrão **Object Mother** é adequado para entidades simples e estáveis, como `User`, com poucas variações previsíveis (padrão e premium). Já o Carrinho é combinatório: usuário diferente, zero ou vários itens, totais distintos. Um `CarrinhoMother` exigiria dezenas de métodos (`umCarrinhoVazio()`, `umCarrinhoPremiumComDoisItens()`, etc.), gerando explosão de métodos e dificultando manutenção. O **Data Builder** resolve isso com API fluente e valores padrão encapsulados.

1.2 Exemplo Antes vs. Depois

Antes — setup manual (Test Smell: Setup Obscuro):

```
const user = new User(2, 'Maria Premium', 'premium@email.com', 'PREMIUM');
const item1 = new Item('Notebook', 150);
const item2 = new Item('Mouse', 50);
const carrinho = new Carrinho(user, [item1, item2]);
```

Depois — setup com Object Mother + Data Builder:

```
const carrinho = new CarrinhoBuilder()
  .comUser(UserMother.umUsuarioPremium())
  .comItens([
    new Item('Notebook', 150),
    new Item('Mouse', 50),
  ])
  .build();
```

O cenário fica explícito: cliente premium com R\$ 200,00 em itens. O leitor identifica rapidamente o que é relevante para aquele teste, sem interpretar linhas de infraestrutura repetidas.

Benefícios: legibilidade no padrão AAA; manutenção centralizada no builder; customização mínima por teste; prevenção de `Obscure Setup` e `Excessive Setup`.

2. Padrões de Test Doubles (Mocks vs. Stubs)

2.1 Teste analisado: cliente Premium finaliza a compra

Dependência	Tipo	Papel
GatewayPagamento	Stub	Retorna { success: true } para permitir o fluxo de sucesso
PedidoRepository	Stub	Retorna pedido salvo com ID 42
EmailService	Mock	Verifica chamada com destinatário, assunto e corpo corretos

2.2 Por que GatewayPagamento é Stub?

O foco é isolar a lógica de negócio do checkout: dado um pagamento aprovado, o serviço aplica desconto premium e conclui o fluxo. O stub **controla o retorno** da cobrança. A asserção `toHaveBeenCalledWith(180, cartaoCredito)` valida o valor cobrado após o desconto de 10% sobre R\$ 200,00 — regra de negócio do SUT, não implementação interna do gateway.

2.3 Por que EmailService é Mock?

O envio de e-mail é efeito colateral. O teste precisa confirmar **se a interação ocorreu** com os parâmetros corretos — Verificação de Comportamento:

- `toHaveBeenCalledTimes(1)` — e-mail enviado uma vez;
- `toHaveBeenCalledWith('premium@email.com', 'Seu Pedido foi Aprovado!', ...)` — dados corretos.

O retorno do e-mail não define sucesso do checkout; o que importa é validar a colaboração entre `CheckoutService` e `EmailService`.

2.4 Cenário complementar: falha no pagamento

No teste de falha, o gateway stub retorna { success: false }. Repository e e-mail são **dummies** (não devem ser invocados). A asserção principal é `expect(pedido).toBeNull()` — Verificação de Estado — confirmando que o fluxo interrompe antes de persistir ou notificar.

3. Suíte implementada

Estrutura entregue no repositório:

```
_tests_/
├── CheckoutService.test.js
├── builders/
│   ├── UserMother.js      (Object Mother)
│   └── CarrinhoBuilder.js  (Data Builder)
```

Cenários cobertos:

- Pagamento recusado → retorna null; sem persistência nem e-mail.
- Cliente padrão → cobrança de R\$ 100,00 sem desconto.
- Cliente premium → cobrança de R\$ 180,00 (10% off) e e-mail de confirmação.

Validação: `npm test` — todos os testes devem passar.

4. Conclusão

O uso deliberado de Object Mother, Data Builder, Stubs e Mocks previne Test Smells desde a escrita dos testes. Builders tornam o setup legível e sustentável; Stubs isolam dependências externas e permitem focar no estado retornado; Mocks documentam contratos de colaboração (como a notificação por e-mail). Em conjunto, esses padrões produzem uma suíte alinhada ao AAA, desacoplada de infraestrutura real e preparada para evolução do código de produção — avançando da correção reativa (refatoração de smells) para a prevenção na origem.